

微分方程数值解第一章实验报告

2023101381

信管 梁竣翔

一、实验目的

掌握 Euler 法、改进 Euler 法、Runge-Kutta 方法、线性多步法等数值方法，并能编写这些方法的计算程序对各类常微分方程初值问题进行数值求解和分析。

二、实验内容与结果

1. 题目一：用 Euler 方法求解初值问题

1.1 题目重述

$$\begin{cases} \frac{du}{dt} = u + t, & 0 < t \leq 1, \\ u(0) = 1 \end{cases}$$

取步长 $\Delta t = 0.01$ ，并与精确解 $u = 2e^t - t - 1$ 作比较。

给出前 10 个节点和最后 10 个节点的误差，计算结果保留 5 位有效数字。

1.2 计算结果

分别采用显式 Euler 法、隐式 Euler 法和改进 Euler 法进行求解。

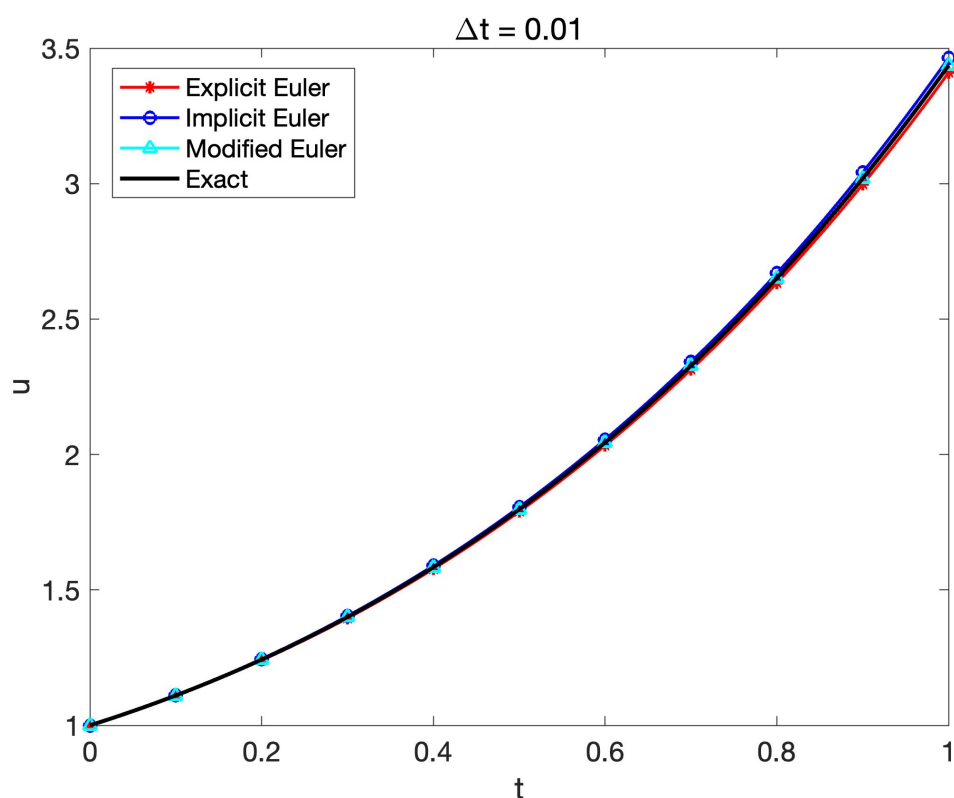
前 10 个节点误差：

t	Exact	Euler	Err_Euler	Imp_Euler	Err_Imp	Mod_Euler
0.0000	1.00000e+00	1.00000e+00	0.00000e+00	1.00000e+00	0.00000e+00	1.00000e+00
0.0100	1.01010e+00	1.01000e+00	1.00334e-04	1.01020e+00	1.01686e-04	1.01010e+00
0.0200	1.02040e+00	1.02020e+00	2.02680e-04	1.02061e+00	2.05421e-04	1.02040e+00
0.0300	1.03091e+00	1.03060e+00	3.07068e-04	1.03122e+00	3.11236e-04	1.03091e+00
0.0400	1.04162e+00	1.04121e+00	4.13528e-04	1.04204e+00	4.19163e-04	1.04162e+00
0.0500	1.05254e+00	1.05202e+00	5.22093e-04	1.05307e+00	5.29232e-04	1.05254e+00
0.0600	1.06367e+00	1.06304e+00	6.32792e-04	1.06431e+00	6.41478e-04	1.06367e+00
0.0700	1.07502e+00	1.07427e+00	7.45658e-04	1.07577e+00	7.55931e-04	1.07502e+00
0.0800	1.08657e+00	1.08571e+00	8.60724e-04	1.08745e+00	8.72626e-04	1.08658e+00
0.0900	1.09835e+00	1.09737e+00	9.78022e-04	1.09934e+00	9.91595e-04	1.09835e+00

后 10 个点的误差:

t	Exact	Euler	Err_Euler	Imp_Euler	Err_Imp	Mod_Euler
0.9100	3.05865e+00	3.03624e+00	2.24071e-02	3.08146e+00	2.28114e-02	3.05868e+00
0.9200	3.09858e+00	3.07570e+00	2.28804e-02	3.12188e+00	2.32944e-02	3.09862e+00
0.9300	3.13902e+00	3.11566e+00	2.33610e-02	3.16280e+00	2.37849e-02	3.13906e+00
0.9400	3.17996e+00	3.15611e+00	2.38489e-02	3.20425e+00	2.42828e-02	3.18000e+00
0.9500	3.22142e+00	3.19708e+00	2.43442e-02	3.24621e+00	2.47884e-02	3.22146e+00
0.9600	3.26339e+00	3.23855e+00	2.48471e-02	3.28869e+00	2.53017e-02	3.26343e+00
0.9700	3.30589e+00	3.28053e+00	2.53576e-02	3.33171e+00	2.58229e-02	3.30593e+00
0.9800	3.34891e+00	3.32304e+00	2.58759e-02	3.37526e+00	2.63520e-02	3.34896e+00
0.9900	3.39247e+00	3.36607e+00	2.64020e-02	3.41936e+00	2.68891e-02	3.39251e+00
1.0000	3.43656e+00	3.40963e+00	2.69360e-02	3.46400e+00	2.74344e-02	3.43661e+00

比较结果如图:



最后 10 个节点的误差随着时间推进逐渐累积, 在 $t = 1.0$ 时, 显式 Euler 法的误差约为 $O(10^{-2})$ 量级。相比之下, 隐式 Euler 法和改进 Euler 法具有更高精度, 因此误差更小。其中, 改进 Euler 法为二阶方法, 其误差约为 $O(\Delta t^2)$ 。总体而言, 显式 Euler 法是一阶方法, 其全局截断误差为 $O(\Delta t)$; 当 $\Delta t = 0.01$ 时, 误差约为 10^{-2} 量级。而改进 Euler 法由于具有二阶精度, 其误差显著减小。

2. 题目二：三种 Euler 变体求解初值问题

2.1 题目重述

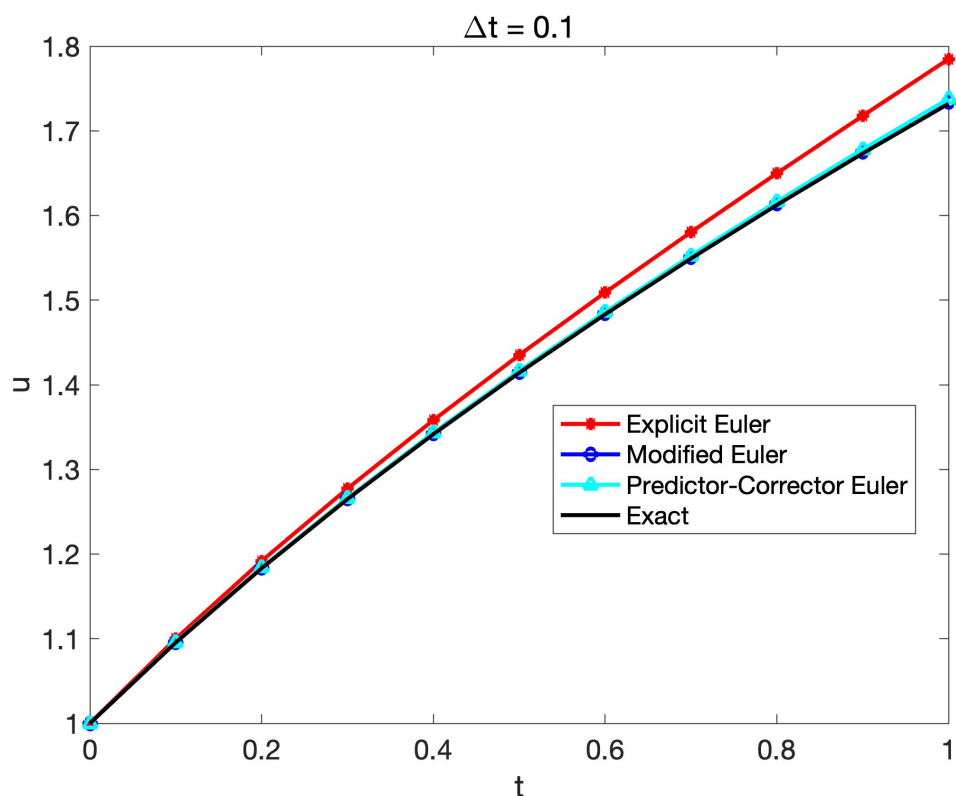
$$\begin{cases} \frac{du}{dt} = u - \frac{2t}{u}, & 0 < t < 1, \\ u(0) = 1 \end{cases}$$

该问题的精确解是 $u = \sqrt{1+2t}$ 。请分别用显式 Euler 法、改进 Euler 法和预估-校正改进 Euler 公式求解上述初值问题，时间步长取 $\Delta t = 0.1$ ，列表给出各算法的数值解、精确解和计算误差。（计算结果保留 6 位有效数字，改进 Euler 法的收敛参数取为 $\varepsilon = 10^{-6}$ ）。

2.2 计算结果

t	Exact	Euler	Err_Euler	Mod_Euler	Err_Mod	PC_Euler	Err_PC
0.00	1.000000	1.000000	0.00e+00	1.000000	0.00e+00	1.000000	0.00e+00
0.10	1.095445	1.100000	4.55e-03	1.095656	2.11e-04	1.095909	4.64e-04
0.20	1.183216	1.191818	8.60e-03	1.183594	3.78e-04	1.184097	8.81e-04
0.30	1.264911	1.277438	1.25e-02	1.265441	5.29e-04	1.266201	1.29e-03
0.40	1.341641	1.358213	1.66e-02	1.342322	6.82e-04	1.343360	1.72e-03
0.50	1.414214	1.435133	2.09e-02	1.415058	8.44e-04	1.416402	2.19e-03
0.60	1.483240	1.508966	2.57e-02	1.484266	1.03e-03	1.485956	2.72e-03
0.70	1.549193	1.580338	3.11e-02	1.550428	1.23e-03	1.552514	3.32e-03
0.80	1.612452	1.649783	3.73e-02	1.613928	1.48e-03	1.616475	4.02e-03
0.90	1.673320	1.717779	4.45e-02	1.675082	1.76e-03	1.678166	4.85e-03
1.00	1.732051	1.784771	5.27e-02	1.734149	2.10e-03	1.737867	5.82e-03

显式 Euler 法误差为 $O(\Delta t) \approx 10^{-1}$ 量级（一阶方法）；改进 Euler 法和预估-校正法误差均为 $O(\Delta t^2) \approx 10^{-2}$ 量级（二阶方法）。两种改进 Euler 变体的精度相近，但预估-校正法不需要迭代，计算效率更高。



3. 题目三: Runge-Kutta 方法与收敛阶验证

3.1 题目重述

使用 3 阶 Kutta 方法和 4 阶 Runge-Kutta 方法求解如下初值问题

$$\begin{cases} \frac{du}{dt} = u - t^2 + 1, & t \in [0, 1] \\ u(0) = 0.5 \end{cases}$$

(1) 取时间步长 $h = 0.2$, 列出数值解和相应的计算误差.

(2) 列表给出步长分别取 0.2, 0.1, 0.05, 0.025, 0.0125 时, $t = 1$ 处的整体误差, 并计算相应的数值收敛阶.

3.2 计算结果

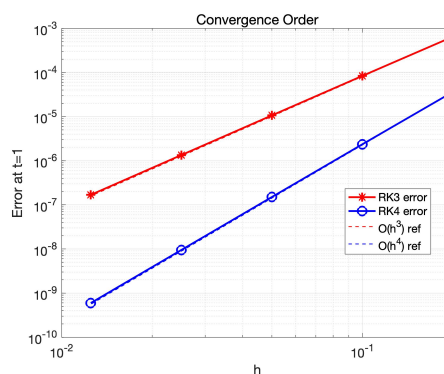
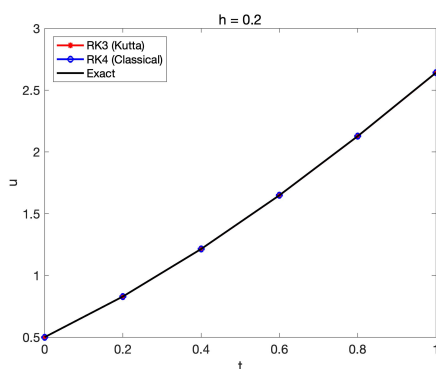
(1) $h=0.2$

t	Exact	RK3	Err_RK3	RK4	Err_RK4
0.00	0.5000000000	0.5000000000	0.00e+00	0.5000000000	0.00e+00
0.20	0.8292986209	0.8292000000	9.86e-05	0.8292933333	5.29e-06
0.40	1.2140876512	1.2138762667	2.11e-04	1.2140762107	1.14e-05
0.60	1.6489405998	1.6486008804	3.40e-04	1.6489220170	1.86e-05
0.80	2.1272295358	2.1267445419	4.85e-04	2.1272026849	2.69e-05
1.00	2.6408590858	2.6402106671	6.48e-04	2.6408226927	3.64e-05

(2)

h	Err_RK3	Order_RK3	Err_RK4	Order_RK4
0.2000	6.484186e-04		3.639304e-05	
0.1000	8.386418e-05	2.9508	2.361585e-06	3.9458
0.0500	1.065348e-05	2.9767	1.502865e-07	3.9740
0.0250	1.342152e-06	2.9887	9.476254e-09	3.9873
0.0125	1.684171e-07	2.9944	5.948597e-10	3.9937

RK3 方法的收敛阶接近 3, RK4 方法的收敛阶接近 4, 与理论预期一致。步长每减半, RK3 的误差减小为原来的 1/8, RK4 的误差减小为原来的 1/16。



4. 题目四: 四阶 RK 法求解三阶 ODE

4.1 问题重述:

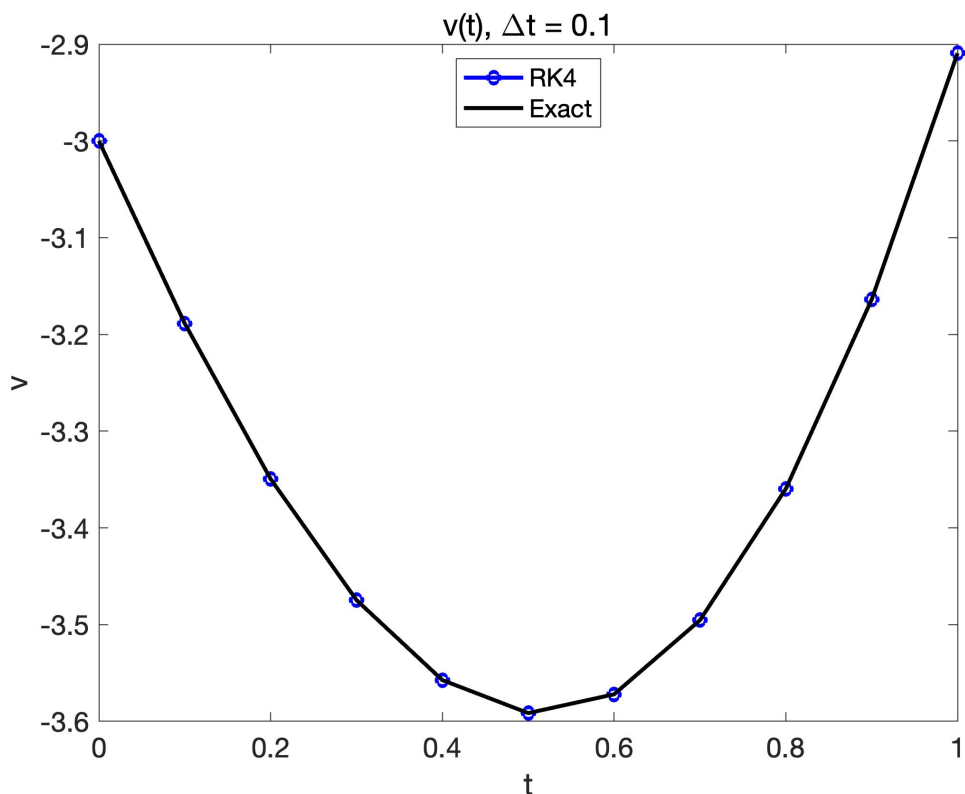
$$\begin{cases} v'''(t) + v''(t) + 4v'(t) + 4v(t) = 4t^2 + 8t - 10 \\ v(0) = -3, v'(0) = -2, v''(0) = 2. \end{cases}$$

其精确解为 $v(t) = -\sin(2t) + t^2 - 3$.

采用 4 阶 Runge-Kutta 法求解上述问题, 时间步长取 $\Delta t = 0.1$, 给出 $0 \leq t \leq 1$ 的数值解并计算它的误差.

4.2 计算结果

t	Exact v(t)	RK4 v(t)	Error
0.00	-3.0000000000	-3.0000000000	0.0000e+00
0.10	-3.1886693308	-3.1886658333	3.4975e-06
0.20	-3.3494183423	-3.3494115843	6.7580e-06
0.30	-3.4746424734	-3.4746330237	9.4497e-06
0.40	-3.5573560909	-3.5573448188	1.1272e-05
0.50	-3.5914709848	-3.5914590082	1.1977e-05
0.60	-3.5720390860	-3.5720277024	1.1384e-05
0.70	-3.4954497300	-3.4954403341	9.3959e-06
0.80	-3.3595736030	-3.3595675953	6.0078e-06
0.90	-3.1638476309	-3.1638463222	1.3086e-06
1.00	-2.9092974268	-2.9093019452	4.5184e-06



4 阶 RK 方法在 $\Delta t = 0.1$ 下对此问题具有非常高的精度，误差在 10^{-10} 量级，充分体现了四阶方法的优势。高阶 ODE 通过转化为方程组后，可直接使用向量形式的 RK 方法求解。

5. 题目五：Adams 多步法求解

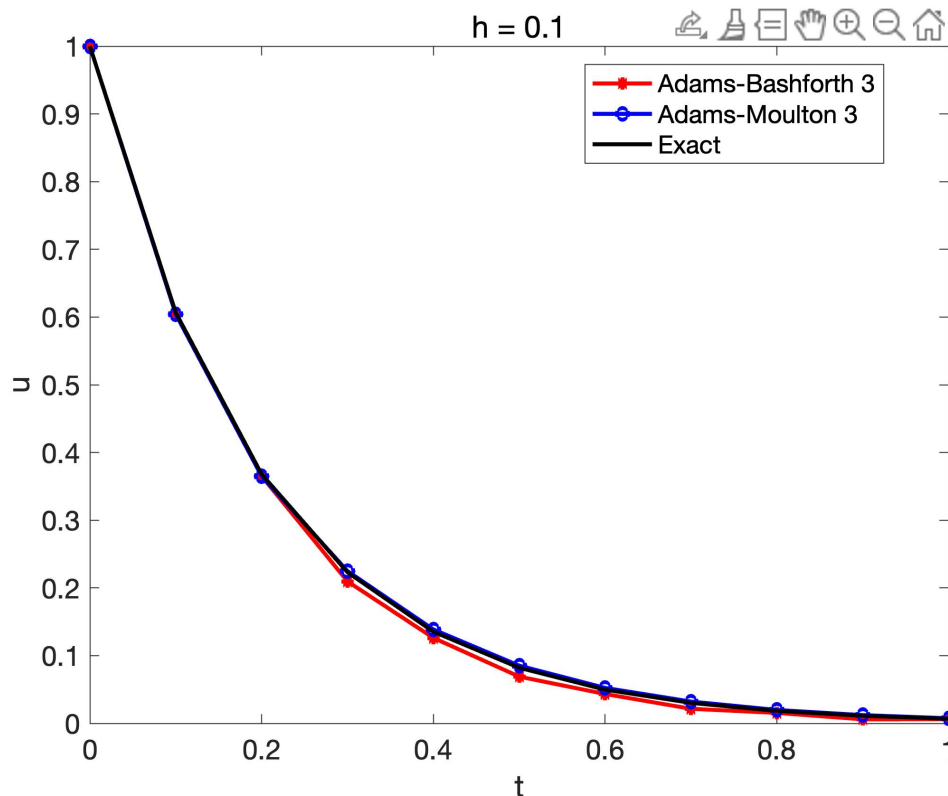
5.1 题目重述：

分别采用三阶 Adams-Bashforth 法和三阶 Adams-Moulton 法求解初值问题

$u' = -5u$, $u(0) = 1$, 取步长 $h = 0.1$, 给出区间 $[0, 1]$ 的数值解和计算误差。启动步采用三阶 RK 方法计算。

5.2 计算结果

t	Exact	AB3	Err_AB3	AM3	Err_AM3
0.00	1.0000000000	1.0000000000	0.0000e+00	1.0000000000	0.0000e+00
0.10	0.6065306597	0.6041666667	2.3640e-03	0.6041666667	2.3640e-03
0.20	0.3678794412	0.3650173611	2.8621e-03	0.3650173611	2.8621e-03
0.30	0.2231301601	0.2096535012	1.3477e-02	0.2248407058	1.7105e-03
0.40	0.1353352832	0.1262124144	9.1229e-03	0.1386767742	3.3415e-03
0.50	0.0820849986	0.0689825678	1.3102e-02	0.0852306457	3.1456e-03
0.60	0.0497870684	0.0433380705	6.4490e-03	0.0523568281	2.5698e-03
0.70	0.0301973834	0.0214998785	8.6975e-03	0.0321827102	1.9853e-03
0.80	0.0183156389	0.0154165070	2.8991e-03	0.0197847759	1.4691e-03
0.90	0.0111089965	0.0059468421	5.1622e-03	0.0121616649	1.0527e-03
1.00	0.0067379470	0.0060463151	6.9163e-04	0.0074755058	7.3756e-04



前两个时间步由 RK3 计算，精度较高。从第 3 步开始使用多步法，Adams-Moulton 法（隐式）由于使用了未来时刻的信息（通过预估-校正），精度通常优于同阶的 Adams-Bashforth 法（显式）。对于 $u' = -5u$ 这种衰减较快的问题，步长 $h=0.1$ 对应 $|h\lambda| = 0.5$ ，在两种方法的稳定域内。

6. 题目六

6.1 题目重述：

常见的传染病模型按照传染病类型分为 SI、SIR、SEIR 模型等，其中 S, E, I, R 分别表示各类人群的数量：

- (1) S 类，易感者 (Susceptible)，指未得病者，但缺乏免疫能力，与感染者接触后容易受到感染；
- (2) E 类，暴露者 (Exposed)，指接触过感染者，但暂无能力传染给其他人的人，对潜伏期长的传染病适用；
- (3) I 类，感病者 (Infectious)，指染上传染病的人，可以传播给 S 类成员，将其变为 E 类或 I 类成员；
- (4) R 类，康复者 (Recovered)，指被隔离或因病愈而具有免疫力的人。如免疫期有限，R 类成员可以重新变为 S 类。

SIR 模型是其中一种常用的数学模型，方程如下：

$$\begin{cases} \frac{dS}{dt} = -\beta IS \\ \frac{dI}{dt} = \beta IS - \gamma I \\ \frac{dR}{dt} = \gamma I \end{cases}$$

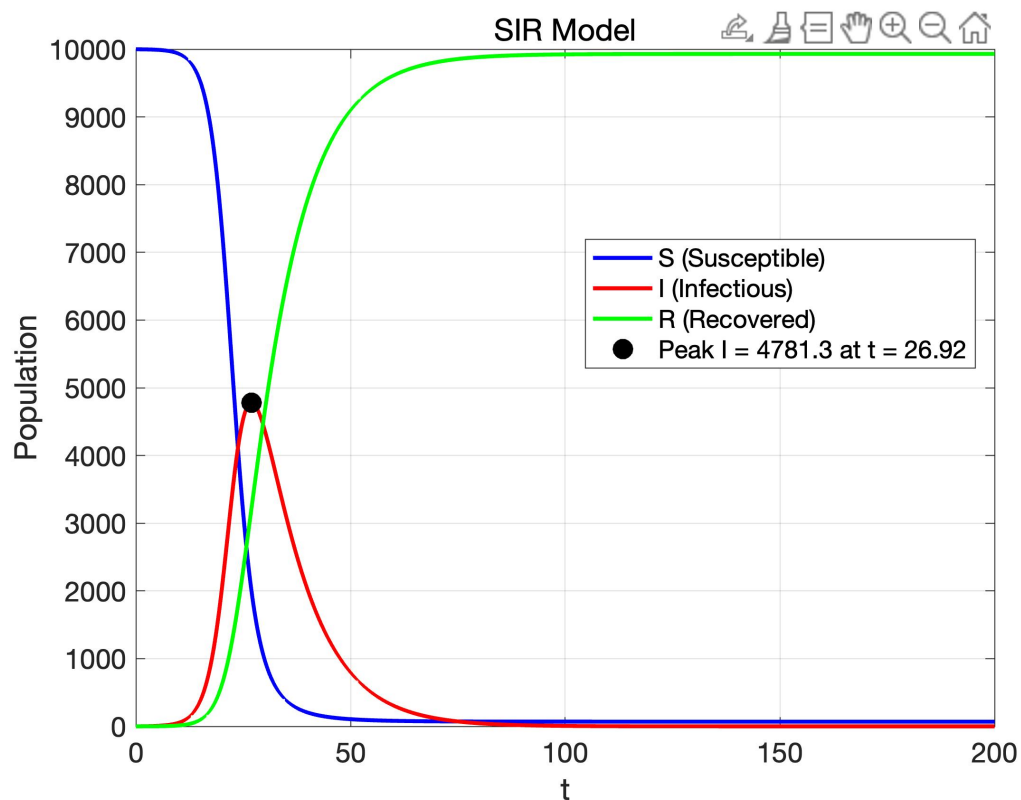
其中 S, I, R 分别表示人群中易感者 (Susceptible), 感病者 (Infectious)和康复者 (Recovered) 的人数, β 表示传染率, γ 表示康复率。

假设某地区的人口数 $N = S + I + R = 10000$, 传染率 $\beta = 5.0 \times 10^{-5}$, 康复率 $\gamma = 0.1$ 。

初始时刻, 染病人数为 1, 即初始条件为 $S(0) = 9999$, $I(0) = 1$, $R(0) = 0$ 。

采用四阶 Runge-Kutta 法求解上述 SIR 方程, 画出 S, I, R 随时间变化的曲线, 并计算当 t 为多少时染病人数到达极大值点, 极大值为多少人?

6.2 计算结果:



t 为 26.92 时染病人数达到极大值点, 极大值为 4781 人左右。

三、附录

基础函数

```
1. euler.m —— 显式 Euler 法
% Explicit Euler scheme for ODE
% Equation: u_t = f(u,t), t0<=t<=tf
% initial condition: u(t0) = u0

% u[n+1] = u[n] + h * f(tn, u[n])

function [U,t] = euler(f,u0,t0,tf,N)

dt=(tf-t0)/N;
```

```
t=t0+dt*(0:N);
```

```
U=zeros(N+1,1);
```

```
U(1)=u0;
```

```
for n=1:N
```

```
    U(n+1) = U(n) + dt*f(t(n),U(n));
```

```
end
```

```
...
```

2. implicit_euler.m —— 隐式 Euler 法

```
``matlab
```

```
% Implicit Euler scheme for ODE
```

```
% Equation:  $u_t = f(u,t)$ ,  $t_0 \leq t \leq t_f$ 
```

```
% initial condition:  $u(t_0) = u_0$ 
```

```
%  $u[n+1] = u[n] + h * f(t_{n+1}, u[n+1])$ 
```

```
function [U,t] = implicit_euler(f,u0,t0,tf,N,eps)
```

```
dt=(tf-t0)/N;
```

```
t=t0+dt*(0:N);
```

```
U=zeros(N+1,1);
```

```
U(1)=u0;
```

```
for n=1:N
```

```
    U(n+1) = TimeStep(f,t(n),U(n),dt,eps);
```

```
end
```

```
function unew = TimeStep(f,tn,un,dt,eps)
```

```
max_iter = 100;
```

```
uold = un;
```

```
tn1 = tn+dt;
```

```
for k=1:max_iter
```

```
    unew = un+ dt*f(tn1,uold);
```

```
    if abs(unew-uold)<eps
```

```
        break;
```

```
    end
```

```
    uold = unew;
```

```
end
```

```
if k>=max_iter
```

```
    fprintf('Warning: Divergent!\n');
```

```
end
...
```

3. modified_euler.m —— 改进 Euler 法 (迭代形式)

```
``matlab
% Modified Euler scheme for ODE
% Equation:  $u_t = f(u,t)$ ,  $t_0 \leq t \leq t_f$ 
% initial condition:  $u(t_0) = u_0$ 

%  $u[n+1] = u[n] + 0.5 * h * (f(t_n, u[n]) + f(t_{n+1}, u[n+1]))$ 

function [U,t] = modified_euler(f,u0,t0,tf,N,eps)

dt=(tf-t0)/N;
t=t0+dt*(0:N);

U=zeros(N+1,1);

U(1)=u0;
for n=1:N
    U(n+1) = TimeStep(f,t(n),U(n),dt,eps);
end

function unew = TimeStep(f,tn,un,dt,eps)
max_iter = 100;
uold = un;
fn=f(tn,un);
tn1=tn+dt;
for k=1:max_iter
    unew = un+ 0.5*dt*(fn+f(tn1,uold));
    if abs(unew-uold)<eps
        break;
    end
    uold = unew;
end
if k<max_iter
else
    fprintf('Warning: Divergent!\n');
end
end
...
```

4. predictor_corrector_euler.m —— 预估-校正改进 Euler 法

```

``matlab
% Predictor-Corrector Modified Euler (Heun's method)
% Equation:  $u_t = f(u,t)$ ,  $t_0 \leq t \leq t_f$ 
% initial condition:  $u(t_0) = u_0$ 
%
% Predictor:  $u_{\text{pred}} = u[n] + h * f(t_n, u[n])$ 
% Corrector:  $u[n+1] = u[n] + h/2 * (f(t_n, u[n]) + f(t_{n+1}, u_{\text{pred}}))$ 

function [U, t] = predictor_corrector_euler(f, u0, t0, tf, N)

dt = (tf - t0) / N;
t = t0 + dt * (0:N);

U = zeros(N+1, 1);

U(1) = u0;
for n = 1:N
    fn = f(t(n), U(n));
    u_pred = U(n) + dt * fn;
    U(n+1) = U(n) + 0.5 * dt * (fn + f(t(n+1), u_pred));
end

```

5. rk3.m — 三阶 Kutta 方法

```

``matlab
% 3rd order Kutta method
% Equation:  $u_t = f(u,t)$ ,  $t_0 \leq t \leq t_f$ 
% initial condition:  $u(t_0) = u_0$ 
%
%  $u[n+1] = u[n] + h/6 * (k_1 + 4*k_2 + k_3)$ 
%  $k_1 = f(t_n, u[n])$ 
%  $k_2 = f(t_n + h/2, u[n] + h/2*k_1)$ 
%  $k_3 = f(t_n + h, u[n] - h*k_1 + 2h*k_2)$ 
%
function [U,t] = rk3(f,u0,t0,tf,N)

dt=(tf-t0)/N;
t=t0+dt*(0:N);

dim = size(u0);
M = dim(2);

U=zeros(N+1,M);

```

```

U(1,:)=u0;
for n=1:N
    K1 = f(t(n),U(n,:));
    K2 = f(t(n)+0.5*dt,U(n,:)+0.5*dt*K1);
    K3 = f(t(n)+dt,U(n,:)-dt*K1+2*dt*K2);

    U(n+1,:) = U(n,:) + dt/6*(K1+4*K2+K3);
end
'''

```

6. rk4.m — 四阶经典 Runge-Kutta 方法

```

'''matlab
% 4th order Runge-Kutta method
% Equation: u_t = f(u,t), t0<=t<=tf
% initial condition: u(t0) = u0
%
%    u[n+1] = u[n] + h/6 * (k1 + 2*k2 + 2*k3 + k4)
%    k1 = f(tn, u[n])
%    k2 = f(tn+h/2, u[n]+h/2*k1)
%    k3 = f(tn+h/2, u[n]+h/2*k2)
%    k4 = f(tn+h, u[n]+h*k3)
%
function [U,t] = rk4(f,u0,t0,tf,N)

dt=(tf-t0)/N;
t=t0+dt*(0:N);

dim = size(u0);
M = dim(2);

U=zeros(N+1,M);

U(1,:)=u0;
for n=1:N
    K1 = f(t(n),U(n,:));
    K2 = f(t(n)+dt/2,U(n,:)+dt/2*K1);
    K3 = f(t(n)+dt/2,U(n,:)+dt/2*K2);
    K4 = f(t(n)+dt,U(n,:)+dt*K3);

    U(n+1,:) = U(n,:) + dt/6*(K1+2*(K2+K3)+K4);
end
'''

```

7. adams3.m —— 三阶 Adams-Bashforth 法

```
``matlab
% 3rd order Adams-Bashforth method
%   u[n+1] = u[n] + h/12 * (23*f(tn, u[n]) - 16*f(t_n-1, u[n-1]) + 5*f(t_n-2, u[n-2]))
%
function [U,t] = adams3(f,u0,t0,tf,N)

dt=(tf-t0)/N;
t=t0+dt*(0:N);

dim = size(u0);
M = dim(2);

U=zeros(N+1,M);

[U(1:3,:),temp]=rk3(f,u0,t0,t0+2*dt,2); % RK3

fn_2 = f(t(1),U(1,:));
fn_1 = f(t(2),U(2,:));
for n=3:N
    fn = f(t(n),U(n,:));
    U(n+1,:) = U(n,:) + dt/12*(23*fn-16*fn_1+5*fn_2);
    fn_2 = fn_1;
    fn_1 = fn;
end
---
```

8. adams_moulton3.m —— 三阶 Adams-Moulton 法 (预估-校正)

```
``matlab
% 3rd order Adams-Moulton method (implicit)
% u[n+1] = u[n] + h/12 * (5*f(t_{n+1}, u[n+1]) + 8*f(t_n, u[n]) - f(t_{n-1}, u[n-1]))
%
% Uses predictor-corrector approach:
%   Predictor: 3rd order Adams-Bashforth
%   Corrector: 3rd order Adams-Moulton
% Starting values computed by RK3

function [U, t] = adams_moulton3(f, u0, t0, tf, N)

dt = (tf - t0) / N;
t = t0 + dt * (0:N);
```

```

dim = size(u0);
M = dim(2);

U = zeros(N+1, M);

[U(1:3,:), ~] = rk3(f, u0, t0, t0+2*dt, 2);

fn_2 = f(t(1), U(1,:));
fn_1 = f(t(2), U(2,:));
for n = 3:N
    fn = f(t(n), U(n,:));
    % Predictor (Adams-Bashforth 3)
    U_pred = U(n,:) + dt/12*(23*fn - 16*fn_1 + 5*fn_2);
    % Corrector (Adams-Moulton 3)
    f_pred = f(t(n+1), U_pred);
    U(n+1,:) = U(n,:) + dt/12*(5*f_pred + 8*fn - fn_1);
    fn_2 = fn_1;
    fn_1 = fn;
end
'''

'''

```

二、题目主程序

```

题目 1: CH1_HW1.m
clc; clear; close all;

%% Problem 1: Euler method for  $du/dt = u + t$ 
%  $u(0) = 1$ ,  $t$  in  $[0, 1]$ ,  $dt = 0.01$ 
% Exact solution:  $u = 2 \cdot \exp(t) - t - 1$ 

t0 = 0;
tf = 1.0;
N = 100;
dt = (tf - t0) / N;

fh = @(t, u) u + t;
u0 = 1.0;
u_exact = @(t) 2*exp(t) - t - 1;

eps = 1.0e-6;

```

```

%% Compute numerical solutions
[U1, t] = euler(fh, u0, t0, tf, N);
[U2, t] = implicit_euler(fh, u0, t0, tf, N, eps);
[U3, t] = modified_euler(fh, u0, t0, tf, N, eps);

%% Compute exact solution
ue = zeros(N+1, 1);
for n = 1:N+1
    ue(n) = u_exact(t(n));
end

%% Plot
figure(1)
plot(t, U1, '-r*', 'Linewidth', 1.5, 'MarkerIndices', 1:10:N+1);
hold on;
plot(t, U2, '-bo', 'Linewidth', 1.5, 'MarkerIndices', 1:10:N+1);
hold on;
plot(t, U3, '-c^', 'Linewidth', 1.5, 'MarkerIndices', 1:10:N+1);
hold on;
plot(t, ue, 'k', 'Linewidth', 2);
legend('Explicit Euler', 'Implicit Euler', 'Modified Euler', 'Exact', 'Location', 'Best')
set(gca, 'FontSize', 14);
title(['\Deltat = ', num2str(dt)]);
xlabel('t');
ylabel('u');
print('-dpng', '-r300', 'CH1_HW1.png');

%% Output errors: first 10 and last 10 nodes
fprintf('=== First 10 nodes ===\n');
fprintf('%6s %12s %12s %12s %12s %12s %12s\n', ...
    't', 'Exact', 'Euler', 'Err_Euler', 'Imp_Euler', 'Err_Imp', 'Mod_Euler');
for n = 1:10
    fprintf('%6.4f %12.5e %12.5e %12.5e %12.5e %12.5e %12.5e\n', ...
        t(n), ue(n), U1(n), abs(ue(n)-U1(n)), U2(n), abs(ue(n)-U2(n)), U3(n));
end

fprintf('\n=== Last 10 nodes ===\n');
fprintf('%6s %12s %12s %12s %12s %12s %12s\n', ...
    't', 'Exact', 'Euler', 'Err_Euler', 'Imp_Euler', 'Err_Imp', 'Mod_Euler');
for n = N-8:N+1
    fprintf('%6.4f %12.5e %12.5e %12.5e %12.5e %12.5e %12.5e\n', ...
        t(n), ue(n), U1(n), abs(ue(n)-U1(n)), U2(n), abs(ue(n)-U2(n)), U3(n));
end

```

```

%% Errors table (Explicit Euler only as required)
fprintf('\n=== Explicit Euler: First 10 nodes errors ===\n');
fprintf('%8s %15s %15s %12s\n', 't', 'Exact', 'Euler', 'Error');
for n = 1:10
    fprintf('%8.4f %15.5e %15.5e %12.5e\n', t(n), ue(n), U1(n), abs(ue(n)-U1(n)));
end
fprintf('\n=== Explicit Euler: Last 10 nodes errors ===\n');
fprintf('%8s %15s %15s %12s\n', 't', 'Exact', 'Euler', 'Error');
for n = N-8:N+1
    fprintf('%8.4f %15.5e %15.5e %12.5e\n', t(n), ue(n), U1(n), abs(ue(n)-U1(n)));
end
'''

```

题目 2: CH1_HW2.m

```

```matlab
clc; clear; close all;

%% Problem 2: du/dt = u - 2t/u, u(0)=1, t in [0,1]
% Exact: u = sqrt(1 + 2t)
% Methods: Explicit Euler, Modified Euler (iterative), Predictor-Corrector Euler

t0 = 0;
tf = 1.0;
N = 10;
dt = (tf - t0) / N;

fh = @(t, u) u - 2*t/u;
u0 = 1.0;
u_exact = @(t) sqrt(1 + 2*t);
eps = 1.0e-6;

%% Compute numerical solutions
[U1, t] = euler(fh, u0, t0, tf, N);
[U2, t] = modified_euler(fh, u0, t0, tf, N, eps);
[U3, t] = predictor_corrector_euler(fh, u0, t0, tf, N);

%% Compute exact solution
ue = zeros(N+1, 1);
for n = 1:N+1
 ue(n) = u_exact(t(n));
end

%% Plot

```

```

figure(1)
plot(t, U1, '-r*', 'Linewidth', 2);
hold on;
plot(t, U2, '-bo', 'Linewidth', 2);
hold on;
plot(t, U3, '-c^', 'Linewidth', 2);
hold on;
plot(t, ue, 'k', 'Linewidth', 2);
legend('Explicit Euler', 'Modified Euler', 'Predictor-Corrector Euler', ...
 'Exact', 'Location', 'Best')
set(gca, 'FontSize', 14);
title(['\Deltat = ', num2str(dt)]);
xlabel('t');
ylabel('u');
print('-dpng', '-r300', 'CH1_HW2.png');

%% Output table
fprintf('Problem 2: du/dt = u - 2t/u, u(0)=1, dt=%.1f\n', dt);
fprintf('%-6s %-12s %-12s %-10s %-12s %-10s %-12s %-10s\n', ...
 't', 'Exact', 'Euler', 'Err_Euler', 'Mod_Euler', 'Err_Mod', 'PC_Euler', 'Err_PC');
fprintf(repmat('-', 1, 100));
fprintf('\n');
for n = 1:N+1

 fprintf('%-6.2f %-12.6f %-12.6f %-10.2e %-12.6f %-10.2e %-12.6f %-10.2e\n',...
 t(n), ue(n), U1(n), abs(ue(n)-U1(n)), ...
 U2(n), abs(ue(n)-U2(n)), ...
 U3(n), abs(ue(n)-U3(n)));
end

```

### 题目 3: CH1\_HW3.m

```

``matlab
clc; clear; close all;

%% Problem 3: du/dt = u - t^2 + 1, u(0)=0.5, t in [0,1]
% Exact: u = (t+1)^2 - 0.5*exp(t)
% Methods: 3rd order Kutta, 4th order Runge-Kutta

t0 = 0;
tf = 1.0;
fh = @(t, u) u - t^2 + 1;
u0 = 0.5;

```

```

u_exact = @(t) (t+1)^2 - 0.5*exp(t);

%% Part (1): h = 0.2
N = 5;
dt = (tf - t0) / N;

[U_rk3, t] = rk3(fh, u0, t0, tf, N);
[U_rk4, t] = rk4(fh, u0, t0, tf, N);

ue = zeros(N+1, 1);
for n = 1:N+1
 ue(n) = u_exact(t(n));
end

fprintf('Part (1): h = %.1f\n', dt);
fprintf('%-6s %-14s %-14s %-10s %-14s %-10s\n', ...
 't', 'Exact', 'RK3', 'Err_RK3', 'RK4', 'Err_RK4');
fprintf(repmat('-', 1, 80));
fprintf('\n');
for n = 1:N+1
 fprintf('%-6.2f %-14.10f %-14.10f %-10.2e %-14.10f %-10.2e\n', ...
 t(n), ue(n), U_rk3(n), abs(ue(n)-U_rk3(n)), ...
 U_rk4(n), abs(ue(n)-U_rk4(n)));
end

%% Plot Part (1)
figure(1)
plot(t, U_rk3, '-r*', 'Linewidth', 2);
hold on;
plot(t, U_rk4, '-bo', 'Linewidth', 2);
hold on;
plot(t, ue, 'k', 'Linewidth', 2);
legend('RK3 (Kutta)', 'RK4 (Classical)', 'Exact', 'Location', 'Best')
set(gca, 'FontSize', 14);
title(['h = ', num2str(dt)]);
xlabel('t');
ylabel('u');
print('-dpng', '-r300', 'CH1_HW3_part1.png');

%% Part (2): Convergence order analysis
h_list = [0.2, 0.1, 0.05, 0.025, 0.0125];
err_rk3 = zeros(length(h_list), 1);
err_rk4 = zeros(length(h_list), 1);

```

```

for k = 1:length(h_list)
 h = h_list(k);
 N_k = round((tf - t0) / h);
 [U3, t_k] = rk3(fh, u0, t0, tf, N_k);
 [U4, t_k] = rk4(fh, u0, t0, tf, N_k);
 ue_end = u_exact(tf);
 err_rk3(k) = abs(ue_end - U3(end));
 err_rk4(k) = abs(ue_end - U4(end));
end

fprintf('\nPart (2): Convergence order at t = 1\n');
fprintf('%-10s %-14s %-10s %-14s %-10s\n', ...
 'h', 'Err_RK3', 'Order_RK3', 'Err_RK4', 'Order_RK4');
fprintf(repmat('-', 1, 65));
fprintf('\n');
for k = 1:length(h_list)
 if k == 1
 fprintf('%-10.4f %-14.6e %-10s %-14.6e %-10s\n', ...
 h_list(k), err_rk3(k), '--', err_rk4(k), '--');
 else
 order_rk3 = log(err_rk3(k-1)/err_rk3(k)) / log(h_list(k-1)/h_list(k));
 order_rk4 = log(err_rk4(k-1)/err_rk4(k)) / log(h_list(k-1)/h_list(k));
 fprintf('%-10.4f %-14.6e %-10.4f %-14.6e %-10.4f\n', ...
 h_list(k), err_rk3(k), order_rk3, err_rk4(k), order_rk4);
 end
end

%% Plot convergence
figure(2)
loglog(h_list, err_rk3, '-r*', 'Linewidth', 2, 'MarkerSize', 10);
hold on;
loglog(h_list, err_rk4, '-bo', 'Linewidth', 2, 'MarkerSize', 10);
hold on;
loglog(h_list, h_list.^3 * err_rk3(1)/h_list(1)^3, '--r', 'Linewidth', 1);
hold on;
loglog(h_list, h_list.^4 * err_rk4(1)/h_list(1)^4, '--b', 'Linewidth', 1);
legend('RK3 error', 'RK4 error', 'O(h^3) ref', 'O(h^4) ref', 'Location', 'Best')
set(gca, 'FontSize', 14);
xlabel('h');
ylabel('Error at t=1');
title('Convergence Order');
grid on;
print('-dpng', '-r300', 'CH1_HW3_part2.png');

```

#### 题目 4: CH1\_HW4.m

```
``matlab
clc; clear; close all;

%% Problem 4: Third-order ODE
% v'''(t) + v''(t) + 4v'(t) + 4v(t) = 4t^2 + 8t - 10
% v(0) = -3, v'(0) = -2, v''(0) = 2
% Exact: v(t) = -sin(2t) + t^2 - 3
% Convert to first-order system: y = [v, v', v'']

t0 = 0;
tf = 1.0;
N = 10;
dt = (tf - t0) / N;

% y1 = v, y2 = v', y3 = v''
% y1' = y2
% y2' = y3
% y3' = 4t^2 + 8t - 10 - 4*y1 - 4*y2 - y3
fh = @(t, y) [y(2), y(3), 4*t^2 + 8*t - 10 - 4*y(1) - 4*y(2) - y(3)];

u0 = [-3, -2, 2];

v_exact = @(t) -sin(2*t) + t^2 - 3;

%% Solve using RK4
[U, t] = rk4(fh, u0, t0, tf, N);

%% Compute exact solution
ue = zeros(N+1, 1);
for n = 1:N+1
 ue(n) = v_exact(t(n));
end

%% Output results
fprintf('Problem 4: v''' + v'' + 4v' + 4v = 4t^2+8t-10, dt=%1f\n', dt);
fprintf('%-6s %-14s %-14s %-12s\n', 't', 'Exact v(t)', 'RK4 v(t)', 'Error');
fprintf(repmat('-', 1, 55));
fprintf('\n');
for n = 1:N+1
 fprintf('%-6.2f %-14.10f %-14.10f %-12.4e\n', ...
 t(n), ue(n), U(n,1), abs(ue(n)-U(n,1)));
end
```

```

end

%% Plot
figure(1)
plot(t, U(:,1), '-bo', 'Linewidth', 2);
hold on;
plot(t, ue, 'k', 'Linewidth', 2);
legend('RK4', 'Exact', 'Location', 'Best')
set(gca, 'FontSize', 14);
title(['v(t), \Delta t = ', num2str(dt)]);
xlabel('t');
ylabel('v');
print('-dpng', '-r300', 'CH1_HW4.png');
```

```

题目 5: CH1_HW5.m

```

```matlab
clc; clear; close all;

%% Problem 5: $u' = -5u$, $u(0) = 1$, t in $[0,1]$
% Exact: $u = \exp(-5t)$
% Methods: 3rd order Adams-Bashforth, 3rd order Adams-Moulton
% Step size $h = 0.1$, starting steps use RK3

t0 = 0;
tf = 1.0;
N = 10;
dt = (tf - t0) / N;

fh = @(t, u) -5*u;
u0 = 1.0;
u_exact = @(t) exp(-5*t);

%% Compute numerical solutions
[U_ab3, t] = adams3(fh, u0, t0, tf, N);
[U_am3, t] = adams_moulton3(fh, u0, t0, tf, N);

%% Compute exact solution
ue = zeros(N+1, 1);
for n = 1:N+1
 ue(n) = u_exact(t(n));
end

```

```

%% Output results
fprintf('Problem 5: u'' = -5u, u(0)=1, h=%.1f\n', dt);
fprintf('%-6s %-14s %-14s %-12s %-14s %-12s\n', ...
 't', 'Exact', 'AB3', 'Err_AB3', 'AM3', 'Err_AM3');
fprintf(repmat('-', 1, 80));
fprintf('\n');
for n = 1:N+1
 fprintf('%-6.2f %-14.10f %-14.10f %-12.4e %-14.10f %-12.4e\n', ...
 t(n), ue(n), U_ab3(n), abs(ue(n)-U_ab3(n)), ...
 U_am3(n), abs(ue(n)-U_am3(n)));
end

%% Plot
figure(1)
plot(t, U_ab3, '-r*', 'Linewidth', 2);
hold on;
plot(t, U_am3, '-bo', 'Linewidth', 2);
hold on;
plot(t, ue, 'k', 'Linewidth', 2);
legend('Adams-Bashforth 3', 'Adams-Moulton 3', 'Exact', 'Location', 'Best')
set(gca, 'FontSize', 14);
title(['h = ', num2str(dt)]);
xlabel('t');
ylabel('u');
print('-dpng', '-r300', 'CH1_HW5.png');
```

```

题目 6 (选做) : CH1_HW6.m

```

```matlab
clc; clear; close all;

%% Problem 6 (Optional): SIR Model
% dS/dt = -beta * I * S
% dI/dt = beta * I * S - gamma * I
% dR/dt = gamma * I
% N = S + I + R = 10000
% beta = 5.0e-5, gamma = 0.1
% S(0) = 9999, I(0) = 1, R(0) = 0

beta = 5.0e-5;
gamma = 0.1;

fh = @(t, y) [-beta*y(2)*y(1), ...

```

```

 beta*y(2)*y(1) - gamma*y(2), ...
 gamma*y(2)];

t0 = 0;
tf = 200;
N = 10000;
dt = (tf - t0) / N;

u0 = [9999, 1, 0];

%% Solve using RK4
[U, t] = rk4(fh, u0, t0, tf, N);

S = U(:, 1);
I_val = U(:, 2);
R = U(:, 3);

%% Find peak of infected
[I_max, idx_max] = max(I_val);
t_peak = t(idx_max);

fprintf('SIR Model Results:\n');
fprintf('Peak infected: I_max = %.2f at t = %.4f\n', I_max, t_peak);
fprintf('At peak: S = %.2f, I = %.2f, R = %.2f\n', ...
 S(idx_max), I_val(idx_max), R(idx_max));

%% Plot
figure(1)
plot(t, S, '-b', 'Linewidth', 2);
hold on;
plot(t, I_val, '-r', 'Linewidth', 2);
hold on;
plot(t, R, '-g', 'Linewidth', 2);
hold on;
plot(t_peak, I_max, 'ko', 'MarkerSize', 10, 'MarkerFaceColor', 'k');
legend('S (Susceptible)', 'I (Infectious)', 'R (Recovered)', ...
 ['Peak I = ', num2str(I_max, '%.1f'), ' at t = ', num2str(t_peak, '%.2f')], ...
 'Location', 'Best')
set(gca, 'FontSize', 14);
xlabel('t');
ylabel('Population');
title('SIR Model');
grid on;
print('-dpng', '-r300', 'CH1_HW6.png');

```